TD3 : API de calcul de trajets

Travail à préparer avant la séance

Ce TD utilise le module de calcul de distances "OpenRouteService". Une documentation de ce module, dédiée à cet enseignement, est disponible ici : `https://rtavenar.github.io/doc_openrouteservice/`

Il est donc nécessaire de se créer un compte avant le début du TD.

— Créer un compte gratuit sur `https://openrouteservice.org/dev/#/signup`

— Se rendre sur la page API key en étant connecté avec son compte.

— Copier la clé "Basic Key" située en haut de la page : elle sera nécessaire pour chaque requête.

Préambule

1. Télécharger sur Cursus l'archive contenant les données sources du TD3
2. Décompresser cette archive sur votre disque. Les fichiers doivent se trouver dans PythonOpenData/TD3.
3. Lancer l'éditeur Visual Studio Code.
4. Dans Visual Studio Code, ouvrir le dossier PythonOpenData/TD3.

Exercice 1 : Préparation de la clé d'API

1. Dans le fichier credentials.json, insérer votre clé d'API préalablement créée.
2. Dans le fichier td3.py, la fonction fournie `acces_cle_api` permet de lire la clé API dans le fichier credentials.json. Utiliser cette fonction pour afficher votre clé d'API.

Exercice 2 : Obtenir des coordonnées GPS

1. Etudier la documentation de l'API de géocodage.
2. Par similitude, obtenir les coordonnées GPS de "Villejean Université Rennes".
3. A partir de cet exemple, écrire une fonction `adresse_vers_gps` qui prend en entrée une clé d'api et une adresse textuelle, et renvoie les coordonnées gps, sous forme d'une chaine de caractères `"longitude,latitude"` (sans espace).
4. Tester cette fonction pour calculer les coordonnées GPS de "Villejean Université Rennes", puis de "Campus Beaulieu Rennes".

Exercice 3 : Calculer la distance d'un trajet

1. Etudier la documentation de l'API d'itinéraires (directions).
2. Écrire une fonction `distance_trajet_coord` qui prend en entrée une clé d'api, les coordonnées gps d'un premier lieu, et les coordonnées gps d'un second lieu. Cette fonction doit renvoyer la distance *en kilomètres*, en voiture, entre les deux lieux. Les coordonnées sont données sous la forme d'une chaine de caractères "longitude,latitude".

   Indice : On pourra s'inspirer largement de la fonction `adresse_vers_gps`. Il faudra changer principalement 3 éléments : l'url de l'api appelée, les paramètres de la requête, la lecture du résultat.

3. En utilisant les coordonnées obtenues précédemment, calculer la distance en voiture de Villejean à Beaulieu, puis de Beaulieu à Villejean.
4. Écrire une fonction `distance_trajet_adresse` qui prend en entrée une clé d'api, l'adresse d'un premier lieu, l'adresse d'un second lieu et qui renvoie la distance *en kilomètres*, en voiture, entre les deux lieux.
5. Tester cette fonction pour calculer la distance de Rennes à Brest. Noter qu'il est probablement nécessaire de préciser "Brest, France".

Exercice 4 : Calculer la durée d'un trajet

1. Écrire une fonction `duree_trajet_coord` qui prend en entrée une clé d'api, les coordonnées gps d'un premier lieu, et les coordonnées gps d'un second lieu. Cette fonction doit renvoyer la duree en minutes, en voiture, entre les deux lieux. Les coordonnées sont données sous la forme d'une chaine de caractères "longitude,latitude".
2. Tester cette fonction pour calculer la durée en voiture de Villejean à Beaulieu.
3. Modifier la fonction `duree_trajet` pour qu'elle prenne un attribut facultatif `mode`, correspondant au mode de transport. Cet attribut vaut `driving-car` par défaut.
4. Afficher la duree pour aller de Villejean à Beaulieu en voiture, en vélo, puis à pieds.
5. Écrire une fonction `duree_trajet_adresse` qui prend en entrée une clé d'api, l'adresse d'un premier lieu, l'adresse d'un second lieu et qui renvoie la durée en minutes en voiture, entre les deux lieux.
6. Tester cette fonction pour calculer la durée d'un trajet entre Rennes et Marseille.

Exercice 5 : Voyage à étapes

On définit un voyage comme une liste de lieux (chaines de caractère). Chaque lieu de la liste est une étape du voyage.

Pour toutes les fonctions demandées, penser à chaque fois à utiliser d'autres fonctions existantes !

1. Une variable `voyage1` est définie dans le fichier source. Définir un second voyage de votre choix.
2. Définir une fonction `distances_etapes` qui prend en entrée une clé d'API, un voyage, et renvoie la liste des distances de chaque étape. Tester sur les deux voyages.
3. Définir une fonction `distance_totale` qui prend en entrée une clé d'API, un voyage, et renvoie la distance totale du trajet en passant par chacune des étapes.